

Understanding the BM25 full text search algorithm

Nov 19, 2024

BM25, or Best Match 25, is a widely used algorithm for full text search. It is the default in Lucene/Elasticsearch and SQLite, among others. Recently, it has become common to combine full text search and vector similarity search into "hybrid search". I wanted to understand how full text search works, and specifically BM25, so here is my attempt at understanding by re-explaining.

1. [Motivation: can BM25 scores be compared across queries?](#)
2. [Ranking documents probabilistically](#)
3. [Components of BM25](#)
4. [Behold, Math!](#)
 1. [Query terms](#)
 2. [Inverse Document Frequency \(IDF\)](#)
 3. [Term frequency in the document](#)
 4. [Document length normalization](#)
 5. [Putting it all together](#)
5. [Cleverness of BM25 and its precursors](#)
 1. [Ranking by probability without calculating probability](#)
 2. [Assuming most documents are irrelevant](#)
6. [Conclusion: BM25 scores can be compared within the same collection](#)
7. [Further reading](#)

Motivation: can BM25 scores be compared across queries?

For a quick bit of context on why I'm thinking about search algorithms, I'm building a personalized content feed that scours noisy sources for content related to your interests. I started off using vector similarity search and wanted to also include full-text search to improve the handling of exact keywords (for example, a friend has "Solid.js" as an interest and using vector similarity search alone, that turns up more content related to React than Solid).

The question that motivated this deep dive into BM25 was: can I compare the BM25 scores of documents across multiple queries to determine which query the document best matches?

Initially, both ChatGPT and Claude told me no — though annoyingly, after doing this deep dive and formulating a more precise question, they both said yes 🤔. Anyway, let's get into the details of BM25 and then I'll share my conclusions about this question.

Ranking documents probabilistically

At the most basic level, the goal of a full text search algorithm is to take a query and find the most relevant documents from a set of possibilities.

However, we don't really know which documents are "relevant", so the best we can do is guess. Specifically, we can rank documents based on the probability that they are relevant to the query. (This is called *The Probability Ranking Principle*.)

How do we calculate the probability that a document is relevant?

For full text or lexical search, we are only going to use qualities of the search query and each of the documents in our collection. (In contrast, vector similarity search might use an embedding model trained on an external corpus of text to represent the meaning or semantics of the query and document.)

Components of BM25

BM25 uses a couple of different components of the query and the set of documents:

- **Query terms:** if a search query is made up of multiple terms, BM25 will calculate a separate score for each term and then sum them up.
- **Inverse Document Frequency (IDF):** how rare is a given search term across the entire document collection? We assume that common words (such as "the" or "and") are less informative than rare words. Therefore, we want to boost the importance of rare words.
- **Term frequency in the document:** how many times does a search term appear in a given document? We assume that more repetition of a query term in a given document increases the likelihood that that document is related to the term. However, BM25 also adjusts this so that there are diminishing returns each time a term is repeated.
- **Document length:** how long is the given document compared to others? Long documents might repeat the search term more, just by virtue of being longer. We don't want to unfairly boost long documents, so BM25 applies some normalization based on how the document's length compares to the average.

These four components are what make up BM25. Now, let's look at exactly how they're used.

Behold, math!

The BM25 algorithm might look scary to non-mathematicians (my eyes glazed over the first time I saw it), but I promise, it's not too hard to understand!

Here is the full equation:

$$\text{score}(D, Q) = \sum_{i=1}^n \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right) \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Now, let's go through it piece-by-piece.

Query terms

$$\text{score}(D, Q) = \sum_{i=1}^n \dots$$

- D is a given document
- Q is the full query, potentially composed of multiple query terms
- n is the number of query terms
- q_i is each of the query terms

This part of the equation says: given a document and a query, sum up the scores for each of the query terms.

Now, let's dig into how we calculate the score for each of the query terms.

Inverse Document Frequency (IDF)

The first component of the score calculates how rare the query term is within the whole collection of documents using the Inverse Document Frequency (IDF).

$$\ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

The key elements to focus on in this equation are:

- N is the total number of documents in our collection
- $n(q_i)$ is the number of documents that contain the query term
- $N - n(q_i)$ therefore is the number of documents that *do not* contain the query term

In simple language, this part boils down to the following: common terms will appear in many documents. If the term appears in many documents, we will have a small number ($N - n(q_i)$, or the number of documents that *do not* have the term) divided by N . As a result, common terms will have a small effect on the score.

In contrast, rare terms will appear in few documents so $n(q_i)$ will be small and $N - n(q_i)$ will be large. Therefore, rare terms will have a greater impact on the score.

The constants 0.5 and 1 are there to smooth out the equation and ensure that we don't end up with wildly varying results if the term is either very rare or very common.

Term frequency in the document

In the previous step, we looked at how rare the term is across the whole set of documents. Now, let's look at how frequent the given query is in the given document.

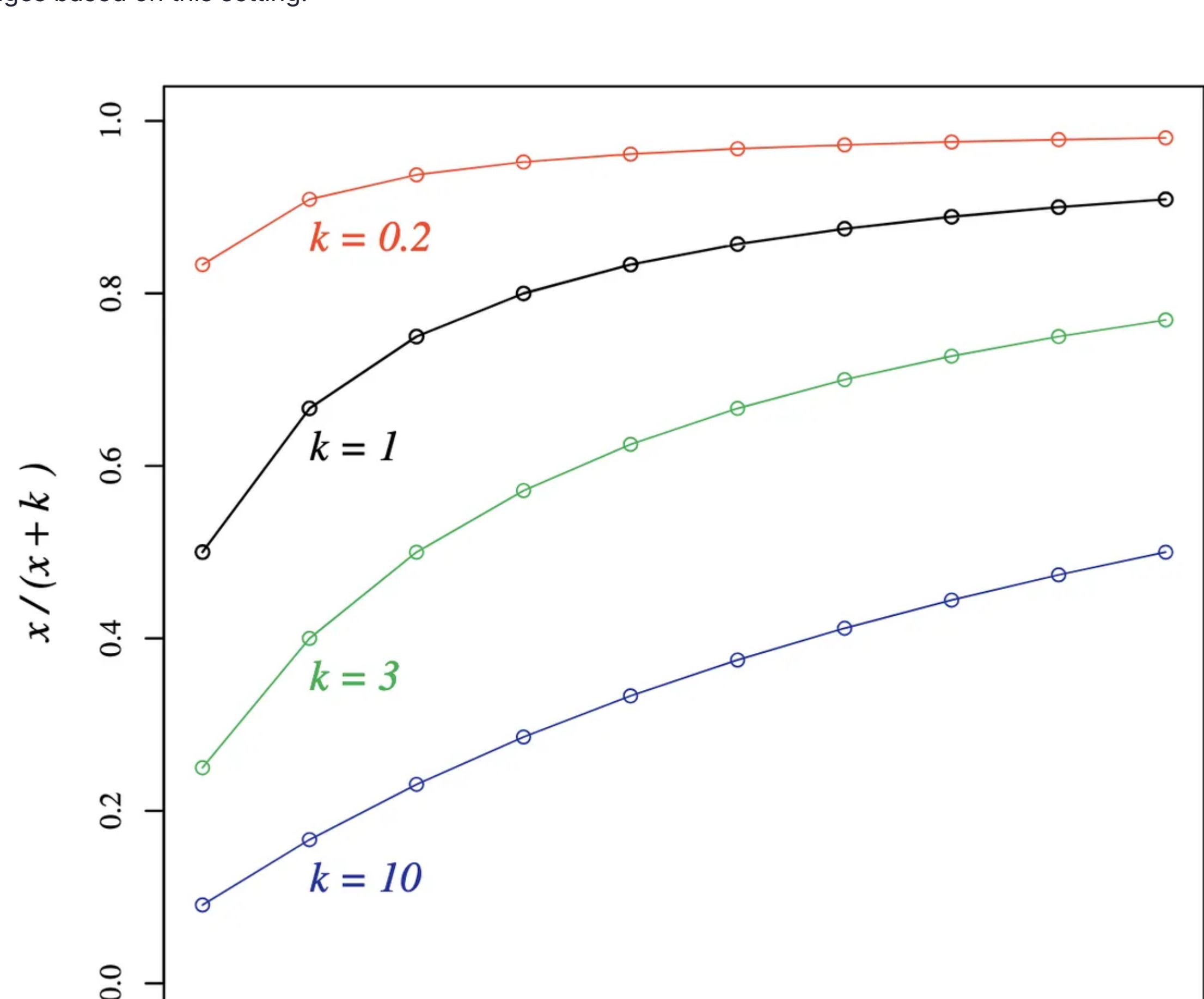
$$\frac{f(q_i, D)}{f(q_i, D) + k_1}$$

The terms in this equation are:

- q_i is a given query
- D is a given document
- $f(q_i, D)$ is the frequency of the given query in the given document
- k_1 is a tuning parameter that is generally set between 1.2 and 2

This equation takes the term frequency within the document into effect, but ensures that term repetition has diminishing returns. The intuition here is that, at some point, the document is probably related to the query term and we don't want an infinite amount of repetition to be weighted too heavily in the score.

The k_1 parameter controls how quickly the returns to term repetition diminish. You can see how the slope changes based on this setting:



From [The Probabilistic Relevance Framework: BM25 and Beyond](#)

Document length normalization

The last thing we need is to compare the length of the given document to the lengths of the other documents in the collection.

$$(1 - b + b \cdot \frac{|D|}{\text{avgdl}})$$

From right to left this time, the parameters are:

- $|D|$ is the length of the given document
- avgdl is the average document length in our collection
- b is another tuning parameter that controls how much we normalize by the document length

Long documents are likely to contain the search term more frequently, just by virtue of being longer. Since we don't want to unfairly boost long documents, this whole term is going to go in the denominator of our final equation. That is, a document that is longer than average ($\frac{|D|}{\text{avgdl}} > 1$) will be penalized by this adjustment.

b can be adjusted by the user. Setting $b = 0$ turns off document length normalization, while setting $b = 1$ applies it fully. It is normally set to 0.75.

Putting it all together

If we take all of the components we've just discussed and put them together, we arrive back at the full BM25 equation:

$$\text{score}(D, Q) = \sum_{i=1}^n \underbrace{\ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)}_{\text{Inverse Document Frequency}} \underbrace{\frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}}_{\text{Document length normalization}}$$

Reading from left to right, you can see that we are summing up the scores for each query term. For each, we are taking the Inverse Document Frequency, multiplying it by the term frequency in the document (with diminishing returns), and then normalizing by the document length.

Cleverness of BM25 and its precursors

We've just gone through the components of the BM25 equation, but I think it's worth pausing to emphasize two of its most ingenious aspects.

Ranking by probability without calculating probability

As mentioned earlier, BM25 is based on an idea called the Probability Ranking Principle. In short, it says:

If retrieved documents are ordered by decreasing probability of relevance on the data available, then the system's effectiveness is the best that can be obtained for the data.

- [The Probabilistic Relevance Framework: BM25 and Beyond](#)

Unfortunately, calculating the "true" probability that a document is relevant to a query is nearly impossible.

However, we really care about the *order* of the documents more than we care about the exact probability. Because of this, researchers realized that you could simplify the equations and make it practicable.

Specifically, you could drop terms from the equation that would be required to calculate the full probability but where leaving them out would not affect the *order*.

Even though we are using the Probability Ranking Principle, we are actually calculating a "weight" instead of a probability.

$$W(d) = \sum_{f_{i,d} > 0} \log \frac{P(F = f_{i,d} | R = 1) P(F = 0 | R = 0)}{P(F = f_{i,d} | R = 0) P(F = 0 | R = 1)}$$

This equation calculates the weight using term frequencies. Specifically:

- $W(d)$ is the weight for a given document
- $P(F = f_{i,d} | R = 1)$ is the probability that the query term would appear in the document with a given frequency ($f_{i,d}$) if the document is relevant ($R = 1$)

The various terms boil down to the probability that we would see a certain query term frequency within the document if the document is relevant or not relevant, and the probabilities that the term would not appear at all if the document is relevant or not.

The Robertson/Sparck Jones Weight is a way of estimating these probabilities but only using the counts of different sets of documents:

$$w^{RSJ} = \log \frac{(r + 0.5)(N - R - r + 0.5)}{(n - r + 0.5)(R - r + 0.5)}$$

The terms here are:

- r is the number of relevant documents that contain the query term
- N is the total number of documents in the collection
- R is the number of relevant documents in the collection
- n is the number of documents that contain the query term

The big, glaring problem with this equation is that you first need to know which documents are relevant to the query. How are we going to do those?

Assuming most documents are irrelevant

The question about how to make use of the Robertson/Joes weight apparently stumped the entire research field for about 15 years. The equation was built up from a solid theoretical foundation, but relying on already having relevance information made it nearly impossible to put to use.

The BM25 developers made a very clever assumption to get to the next step.

For any given query, we can assume that most documents are not going to be relevant. If we assume that the number of relevant documents is so small as to be negligible, we can just set those numbers to zero!

$$R = r = 0$$

If we substitute this into the Robertson/Sparck Jones Weight equation, we get nearly the IDF term used in BM25:

$$\log \frac{(0 + 0.5)(N - 0 - 0 + 0.5)}{(n - 0 + 0.5)(0 - 0 + 0.5)} = \log \frac{0.5(N - 0 + 0.5)}{(n + 0.5)0.5} = \log \frac{(N - 0 + 0.5)}{(n + 0.5)}$$

Not relying on relevance information made BM25 much more useful, while keeping the same theoretical underpinnings. Victor Lavrenko described this as a "[very impressive leap of faith](#)", and I think this is quite a neat bit of BM25's backstory.

Conclusion: BM25 scores can be compared within the same collection

As I mentioned at the start, my motivating question was whether I could compare BM25 scores for a document across queries to understand which query the document best matches.

In general, BM25 scores cannot be directly compared (and this is what ChatGPT and Claude stressed to me in response to my initial inquiries 🤔). The algorithm does not produce a score from 0 to 1 that is easy to compare across systems, and it doesn't even try to estimate the probability that a document is relevant. It only focuses on ranking documents within a certain collection in an order that approximates the probability of their relevance to the query. A higher BM25 score means the document is likely to be *more relevant*, but it isn't the actual probability that it is relevant.

As far as I understand now, it is possible to compare the BM25 scores across queries *for the same document within the same collection of documents*.

My hint that this was the case was the fact that the BM25 sums the scores of each query term. There should not be a semantic difference between comparing the BM25 scores for two query terms and two whole queries.

The important caveat to stress, however, is the *same document within the same collection*. BM25 uses the IDF or rarity of terms as well as the average document length within the collection. Therefore, you cannot necessarily compare scores across time because any modifications to the overall collection could change the scores.

For my purposes, though, this is useful enough. It means that I can do a full text search for each of a user's interests in my collection of content and compare the BM25 scores to help determine which pieces best match their interests.

I'll write more about ranking algorithms and how I'm using the relevance scores in future posts, but in the meantime I hope you've found this background on BM25 useful or interesting!

Thanks to Alex Kesling and Natan Last for feedback on drafts of this post.

Further reading

If you are interested in diving further into the theory and history of BM25, I would highly recommend watching Elastic engineer Britta Weber's 2016 talk [Improved Text Scoring with BM25](#) and reading [The Probabilistic Relevance Framework: BM25 and Beyond](#) by Stephen Robertson and Hugo Zaragoza.

Also, I had initially included comparisons between BM25 and some other algorithms in this post. But, as you know, it was already a bit long 🤔. So, you can now find those in this other post: [Comparing full text search algorithms: BM25, TF-IDF, and Postgres](#).

Discuss on [Lobsters](#) and [Hacker News](#).

[#scour](#) [#search](#) [#understanding](#)



Try [Scour](#)! Get a feed of great content tailored to your interests for free in < 1 minute.